

Thesis title

Abstract

The objective of this project is to help players during a game of 9-ball billiards by showing them how the white ball would likely move after a shot. The system has two main parts: 1. Phone App on the Cue Stick — the app that uses the phone's motion sensors (like gyroscope and accelerometer) to track how the cue stick is held and moved. The app sends this data in real time to a computer. 2. Camera Above the Table + Computer — A camera above the billiard table sends live video to a computer. There, a program detects where all the balls are based on their colors. It figures out where the cue ball is and updates the layout of the table. Both of those parts are essential, because accurate aiming without a good stroke is pointless. And same goes for a great stroke without proper aiming.

Key words

Computer Vision, Billiards, Trajectory Prediction, Augmented Reality, Object Detection

Tytuł pracy

polish

Streszczenie

polish Celem niniejszego projektu jest wspomaganie graczy podczas rozgrywki w bilard (odmiana 9-bil) poprzez wizualizację przewidywanego toru ruchu białej bili po uderzeniu. System składa się z dwóch głównych elementów: 1. Aplikacja mobilna na kiju bilardowym — aplikacja wykorzystuje czujniki ruchu telefonu (takie jak żyroskop i akcelerometr) do śledzenia sposobu trzymania kija oraz jego ruchu. Aplikacja przesyła te dane w czasie rzeczywistym do komputera. 2. Kamera nad stołem i komputer — kamera umieszczona nad stołem bilardowym przesyła obraz na żywo do komputera. Następnie program komputerowy wykrywa położenie wszystkich bil na podstawie ich kolorów, lokalizuje bilę białą i aktualizuje odwzorowanie układu stołu. Oba te elementy są kluczowe, ponieważ precyzyjne celowanie bez poprawnego technicznie uderzenia jest bezcelowe. Analogicznie, nawet idealne uderzenie nie przyniesie efektu bez właściwego wycelowania.

Słowa kluczowe

polish Wizja komputerowa, Bilard, Predykcja trajektorii, Rzeczywistość rozszerzona, Wykrywanie obiektów

Introduction

Billiards, especially 9-ball, is a sport that demands a high level of precision, spatial awareness, and consistent motor control. Unlike many other sports where

physical part is the most important, billiards is fundamentally a game that uses geometry and physics executed through the trained muscle memory.

1 Introduction to the Problem Domain

Game of billiards focuses mainly on two skill sets: the ability to see the correct path of the balls and the physical ability to make the stroke that is needed to shoot the cue ball along that path. New players often have problem with the idea of the "Ghost Ball" [?] — the imaginary position where the cue ball must hit the object ball to pocket it.

Additionally, the way a player executes the stroke — including stance, bridge stability, and wrist movement can have a strong effect on the shot result. Even if a player aims correctly, but introduces unwanted lateral acceleration or English (spin) [?], it can result in a miss. With the development of accessible Computer Vision (CV), we are now able to better digitize these physical interactions to provide live feedback [?].

2 Settling of the Problem in the Domain

Training methods that we have right now are mostly manual, relying on observations of coaches or player's own intuition. We do have professional tracking systems, but they are often quite expensive and need fixed, industrial-grade hardware. Like hawk-eye [?] technology for example. When it comes to mobile applications, they often rely solely on 2D video analysis, which does not have the depth perception that is needed for accurate table mapping or the high-frequency sampling that would be needed for detailed stroke analysis. One of those applications would be "Billard Aiming Calculator" [?].

3 Objective of the Thesis

The main goal of this thesis is to design, implement, and test a "Smart Pool Assistant and Stroke Analyzer". The system would aim to assist a player in real-time by visualizing shot outcomes and showing the analysis of the their cue strokes.

The primary objectives are defined as follows:

- To develop a Computer Vision module that is able to detect billiard balls and the cue stick in a live video feed.
- To implement a physics engine that calculates and visualizes the predicted trajectory of the cue ball (Tangent and Normal lines) based on the "Ghost Ball" principle.
- To create a mobile telemetry system that captures high-frequency accelerometer and gyroscope data from the player's phone.

- To derive important biomechanical metrics, such as the impact force $F = ma$ (where F , m , and a denote force, mass, and acceleration, respectively) and wrist stability, to provide insights on the player's technique.

4 Scope of the Thesis

This project covers the software and hardware needed to build a working prototype for the game of 9-ball.

- **Vision System:** The visual analysis uses only a top-down camera view. The software is written with Python and OpenCV, and uses Roboflow machine learning models for object detection. The trajectory prediction assumes perfect elastic collisions and focuses on the immediate paths of the cue ball and the target ball.
- **Sensor System:** The biomechanical part uses an Android smartphone sensors (Linear Acceleration and Gyroscope). This part also includes creating a TCP/IP protocol to send the data to the computer for processing.
- **Hardware:** The phone is connected to the computer via USB tethering. The mobile app is developed in Java using Android Studio.

5 Short Description of Chapters

The thesis is organized as follows:

Chapter 2 (Problem Analysis) shows the theoretical and mathematical background for the project. It shows the physics behind collisions, the geometry of "Ghost Ball" aiming and the mechanics of sensors. It also presents solutions that already exist in sports technology to set the context.

Chapter 3 (Requirements and Tools) defines the system's functional and non-functional requirements. It also presents the chosen technology stack, and motivates the use of Python, OpenCV, and Android platform.

Chapter 4 (External Specification) describes the system from the user's perspective. It shows the Android App interface and the visualization of the desktop interface. It also explains how the user interacts with the system.

Chapter 5 (Internal Specification) details the technical implementation and backend logic. It explains the algorithms that were used for object detection, trajectories prediction, TCP/IP communication, and sensor data processing.

Chapter 6 (Verification and Validation) presents testing results. Evaluates the computer vision model accuracy, the latency of live data streaming and how reliable are stroke analysis parameters through unit tests and real-world examples.

Chapter 7 (Conclusions) summarizes the thesis, discussing the achieved objectives and suggesting future improvements. Problem Analysis

This chapter presents the theoretical and mathematical basis of the system. It explains the geometric challenges of aiming that were implemented in the vision module and shows the biomechanical principles behind the sensor analysis. It also reviews the solutions to these challenges that already exist.

6 Problem Analysis

The game of 9-ball billiards provides a singular challenge in which a player must convert a 3D intention into a 2D geometric action. The main problems that are addressed by this project are divided into visual perception and biomechanical execution.

6.1 The Visual Paradox (Ghost Ball)

According to Alciatore, the "Ghost Ball" concept is fundamental to understanding how to aim in billiards [?]. The player must visualize an imaginary position where the cue ball must contact the object ball to send it into the pocket. It is often hard to visualize it, because the aiming line, which passes through the ghost ball's center, is not the same as the actual contact point on the object ball. This mismatch requires players to aim at an invisible target, which creates a "visual paradox", that is especially hard for new players without any visual feedback.

6.2 Biomechanical Inconsistency

Even if an aiming line is correctly visualized, if the stroke is poorly executed, it can cause the shot to miss. Therefore, the mechanics of the cue stroke are also really important.

Ballistic vs. Tetanic Stroke: According to Moore [?], there are two main types of strokes in billiards. Amateur players usually use "ballistic" strokes, which are marked by a sudden pull at the start of the movement. It makes the speed control at impact unreliable. In contrast, professional players aim for a "tetanic" stroke, which is characterized by maintaining a continuous muscle contraction that produces smooth, steady acceleration throughout the cue delivery [?].

Lateral Deviation: Another important thing is the wrist stability during the stroke. If the player is not able to maintain a vertical "pendulum swing" by keeping the elbow fixed, the cue tip can move away from the intended aiming line [?]. The most common errors are sideways wrist movement and inconsistent impact force.

7 State of the Art

The field of "Smart Sports" has significantly advanced in the past decades, with the use of Convolutional Neural Networks (CNN) and Wearable Sensors [?].

8 Existing Solutions

- **Hawk-Eye (Computer Vision):** Widely used in sports like snooker or tennis, this system uses multiple calibrated high-speed cameras to determine 3D ball positions. While being highly accurate (< 3 mm error), it requires expensive, fixed infrastructure and it does not analyze player biomechanics, but only ball trajectories [?].
- **Projection AR (Augmented Reality) Systems:** Solutions such as *PoolLiveAid* use overhead projectors to overlay aiming lines directly onto the table. Even though it offers a really nice user experience, it requires expensive hardware and complex calibration [?].
- **Wearable IMU (Inertial Measurement Unit) Analyzers:** There are devices such as *Blast Motion*, that are used in golf or baseball. They use inertial sensors to measure swing parameters. However, they do not analyze anything besides the swing. They monitor the movement, but they do not care about the game environment. For example, they do not analyze the position of the balls [?].

9 Mathematical Models

The implementation of the system is based on mathematical models derived from vector algebra and Newtonian mechanics. These models help describe how objects move and interact with each other. Thanks to this approach, the system can estimate object trajectories and basic physical behavior in a clear and understandable way.

9.1 Ghost Ball Vector Math

The vision module calculates where the aiming vector would likely intersect the target's ball collision zone.

Let P_{cue} and P_{target} denote the centers of the cue ball and the target ball, and let $\vec{v}_{aim}$ be the normalized direction vector of the cue stick. The projection length L_{proj} of the target vector onto the aiming direction is computed using the dot product:

$$L_{proj} = (P_{target} - P_{cue}) \cdot \vec{v}_{aim}. \quad (1)$$

The point on the aiming line closest to the target ball, denoted as P_{close} , is given by:

$$P_{close} = P_{cue} + \vec{v}_{aim} \cdot L_{proj}. \quad (2)$$

A collision is valid only if the perpendicular distance $d_{\perp}$ is smaller than the ball diameter D :

$$d_{\perp} = \|P_{target} - P_{close}\|. \quad (3)$$

If $d_{\perp} < D$, the Ghost Ball position P_{ghost} is obtained by moving back from P_{close} by an offset computed using the Pythagorean theorem:

$$P_{ghost} = P_{close} - \vec{v}_{aim} \cdot \sqrt{D^2 - d_{\perp}^2}. \quad (4)$$

Thanks to this formula, the system draws the "Ghost Ball" circle at the exact position that the cue ball will be at the moment of impact.

9.2 Physics of the Stroke (Sensor Fusion)

The sensor module processes raw data to estimate the force of the shot. Smart-phone sensor data is processed using essential physical principles to transform raw motion into a tangible measure of impact force. This ensures that the representation of stroke strength remains intuitive and reliable, irrespective of individual variations in technique.

9.2.1 Linear Acceleration

The Android `TYPE_LINEAR_ACCELERATION` sensor is used instead of `TYPE_ACCELEROMETER` to avoid the influence of gravity on user's movement. The resulting acceleration vector $\vec{a} = [a_x, a_y, a_z]$ represents the cue stick's actual acceleration. Its magnitude is given by:

$$\|\vec{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}. \quad (5)$$

9.2.2 Force Estimation

To provide the player with a metric of "Shot Power" in Newtons, the system applies Newton's Second Law. Instead of relying on a fixed constant for the arm mass, the system dynamically calculates the effective mass of the player's arm (m_{arm}) based on the user's body weight and gender defined in the `config.json` file.

The effective mass is derived by summing the segmental weights of the hand, forearm, and upper arm using anthropometric coefficients from Plagenhoef et al. [?]. Consequently, the impact force F is determined by substituting the personalized mass into the motion equation:

$$F = m_{arm} \cdot \|\vec{a}_{impact}\| = (W_{body} \cdot \mu_{gender}) \cdot \|\vec{a}_{impact}\|, \quad (6)$$

where:

- m_{arm} denotes the calculated effective arm mass,
- W_{body} is the player's total body weight,

- μ_{gender} is the effective arm mass coefficient (5.77% for males, 4.97% for females) derived from anthropometric data [?],
- $\|\vec{a}_{impact}\|$ represents the magnitude of the acceleration vector at the moment of impact.

10 Algorithms

Two primary algorithmic domains are utilized to interpret the raw data streams: Computer Vision for game state analysis and Signal Processing for biomechanical analysis. Computer Vision is responsible for analyzing the game state by processing the video feed and identifying key objects such as the balls, and the cue stick. Signal Processing is applied to sensor data to detect important physical events, especially the moment of impact.

10.1 Computer Vision Algorithms

The vision module utilizes the YOLO (You Only Look Once) [?] architecture, deployed in two distinct configurations to handle different tracking requirements.

10.1.1 Object Detection (Balls)

To identify the game elements, a standard object detection model based on the YOLOv12 [?] architecture is employed. It classifies objects into categories such as `cue-ball` and `other` (object balls). The inference function f_{det} transforms the input image I into a set of bounding boxes B :

$$f_{det}(I) \rightarrow \{(x, y, w, h, class, conf)_i\}, \quad (7)$$

where (x, y) is the center of the ball, used as the input for the “Ghost Ball” geometric calculation.

10.1.2 Pose Estimation (Cue Stick)

Determining the aiming line requires more precision than a bounding box can provide. Therefore, a **Pose Estimation** model (specifically `yolo8-pose` [?]) is used to detect the cue stick. Instead of a box, this model detects exact semantic keypoints: the `tip` (cue tip) and the `handle` (grip point). The output is a set of coordinates K :

$$f_{pose}(I) \rightarrow \{(x_{tip}, y_{tip}), (x_{handle}, y_{handle})\}. \quad (8)$$

The aiming vector $\vec{v}_{aim}$ is then derived directly from these keypoints:

$$\vec{v}_{aim} = (x_{tip} - x_{handle}, y_{tip} - y_{handle}). \quad (9)$$

10.2 Peak Detection Algorithm

To automatically detect the moment of contact without external triggers, the system implements a real-time peak detection algorithm on the linear acceleration data stream. A hit is registered at time t if the acceleration magnitude $a(t)$ satisfies both a threshold condition and a local maximum condition:

$$a(t) > T_{peak} \quad \wedge \quad a(t) \geq a(t-1) \quad \wedge \quad a(t) \geq a(t+1), \quad (10)$$

where T_{peak} is the noise gate threshold (set to 15.0 m/s^2 in the implementation). This makes sure that the system captures the exact moment of maximum force exertion (F_{max}) for the biomechanical analysis. Requirements and Tools

This chapter details the functional and non-functional requirements and presents the technological stack selected for implementation. It also describes the system's use cases and the method adopted for the design and development process.

11 Functional and Non-functional Requirements

The system requirements were defined to address the problems identified in the previous chapter, specifically the “Ghost Ball” visualization and the biomechanical analysis of the stroke.

11.1 Functional Requirements

The functional requirements define the specific behaviors and functions the system must support. They are categorized by a module:

Vision Module (Desktop):

- **FR-01 Video acquisition:** The system must capture a real-time video stream from a webcam positioned above the billiard table.
- **FR-02 Object detection:** The system must detect and classify the cue ball and object balls.
- **FR-03 Pose estimation:** The system must identify keypoints of the cue stick (tip and handle) to determine the aiming vector.
- **FR-04 Trajectory prediction:** The system must calculate the “Ghost Ball” position and predict the trajectories of the cue ball and target ball based on geometric rules.
- **FR-05 AR visualization:** The system must overlay the predicted paths and the ghost ball indicator onto the video feed in real-time.

Sensor Module (Mobile):

- **FR-06 Data acquisition:** The mobile application must read data from the accelerometer (linear acceleration) and gyroscope at a frequency of at least 50 Hz.

- **FR-07 Data transmission:** The mobile app must transmit sensor data to the desktop server via a TCP/IP socket connection (USB tethering).
- **FR-08 Stroke analysis:** The system must detect the moment of impact (peak acceleration) and calculate the impact force ($F = ma$) and the wrist rotation stability.
- **FR-09 Accuracy:** The “Ghost Ball” projection error should be less than 5% of the ball diameter to be practically useful for aiming.

11.2 Non-functional Requirements

The non-functional requirements define the quality attributes of the system:

- **NFR-01 Real-time performance:** The vision processing pipeline must maintain a frame rate of at least 20 fps (frames per second) to provide smooth visual feedback.
- **NFR-02 Latency:** The latency between the physical cue movement and the AR update should be less than 150 ms to ensure the user perceives the lines as responsive.
- **NFR-03 Sensor synchronization:** The time drift between the video impact detection and sensor peak detection must be handled to correctly associate the physical stroke with the visual event.
- **NFR-04 Usability:** The setup process (camera alignment, phone connection) should be performable by a single user within 5 minutes.

12 Use Cases

The interaction between the user (player) and the system is shown by a number of essential use cases. The primary actor is the player, who interacts with both the physical equipment (phone, cue) and the software (desktop app).

The main use cases are defined as follows:

1. **UC-1 System setup:** The player connects the Android phone to the PC (via USB/ADB), starts the server script, and launches the mobile app. The connection is established if the “connected” status appears.
2. **UC-2 Aiming assistance:** The player addresses the cue ball. The system detects the cue stick, calculates the trajectory, and projects the “Ghost Ball” and aiming lines. The player adjusts their stance based on this visual feedback.
3. **UC-3 Stroke execution & analysis:** The player executes the shot. The system automatically detects the impact via the sensor stream, captures a snapshot of the metrics (force, rotation), and saves the data to a history log.

4. **UC-4 Review history:** The player views the saved “hit history” (csv/images) to analyze their consistency over the training session.

13 Description of Tools

The project applies a modern technology stack combining computer vision, AI (Artificial Intelligence), and mobile development.

13.1 Hardware Tools

- **Webcam:** A standard HD webcam is used for video input. It is mounted in a “top-down” configuration to minimize perspective distortion.
- **Android smartphone:** A device equipped with an IMU. It works as the telemetry unit attached to the player’s cue.
- **Workstation (PC):** A computer with access to the internet and with installed Python 3.x environment to run the vision server and process data.

13.2 Software Tools

- **Python 3.x:** The main programming language for the backend server and vision processing [?].
- **OpenCV (Open Source Computer Vision Library):** Used for image pre-processing, drawing the AR visualization (lines, circles), and rendering the live telemetry graphs [?].
- **Ultralytics YOLO (You Only Look Once):** A state-of-the-art framework for object detection.
 - *yolov12 (detection)*: It is used for robust detection of billiard balls under different lighting conditions [?].
 - *yolov8-pose (keypoint estimation)*: It is specifically used to detect the **tip** and **handle** of the cue stick, allowing accurate vector calculation [?].
- **Roboflow:** A platform used for dataset management, image annotation, and versioning. It facilitated the preparation of the training data for the custom billiard model [?].
- **Android Studio & Java:** The development environment for the mobile sensor application. Java was chosen for its native support of Android sensor APIs [?].
- **ADB (Android Debug Bridge):** It is utilized for establishing a low-latency reverse TCP connection (`adb reverse`) between the Android device and the localhost server [?].

14 Methodology of Design and Implementation

The development followed an iterative prototyping methodology [?], which is suitable for systems that include experimental algorithms (like computer vision) and hardware integration. The process was divided into four phases:

14.1 Phase 1: Data Collection and Model Training

The initial phase focused on the preparation of the Computer Vision module. To ensure the model's reliability and ability to generalize across different environments three datasets were used. One was a collection of private images captured on both standard club-size table and smaller home table and two were open-source datasets available via Roboflow Universe.

For the **Ball Detection** model, the custom dataset was augmented with the "8-pool Dataset" [?]. Similarly, the **Cue Detection** model combined images with the "cue detection Dataset" [?]. Both external datasets are licensed under the Creative Commons 4.0 (CC BY 4.0) license.

To strictly follow machine learning best practices, the final aggregated datasets were partitioned using a fixed split ratio:

- **70% Training Set:** Used for model learning and weight adjustment.
- **20% Validation Set:** Used for hyperparameter tuning and threshold selection (as described in Chapter 6).
- **10% Test Set:** Held out completely to evaluate the final performance metrics.

As shown in the dataset analysis (Figure ??), the data reflects the natural logic of the game. Specifically, chart (a) highlights the significant class imbalance: the "other" class (object balls 1-15) is approximately 15 times more frequent than the "cue ball" class.

Chart (b) visualizes all bounding boxes in the dataset centered at one central point. Since the camera is mounted in a fixed top-down position and all billiard balls have the same physical size, every detected object occupies roughly the same number of pixels. Therefore, all bounding boxes have nearly identical width and height, forming a single, square shape on the plot.

The spatial distribution (c) confirms that balls are clustered within the table area, while the size correlation (d) indicates consistent object dimensions across the dataset.

To reduce possible overfitting as well as improve model dependability under different lighting conditions, image augmentation procedures (including rotation, brightness adjustment, and noise augmentation) were applied during the preprocessing stage.

A similar process was followed for the **Cue Detection model**, where the dataset was annotated with keypoints (skeleton) to define the tip and handle positions for the pose estimation task. The class distribution for this dataset (Figure ??) is uniform as it focuses on a single class, "cue".

14.2 Phase 2: Vision Logic Implementation

Once the models were trained, the Python logic was developed to translate raw detections into the game state information. This involved implementing the vector algebra for the “Ghost Ball” [?] algorithm and using OpenCV [?] to render the visual overlays.

14.3 Phase 3: Sensor Module Development

The Android application was developed to reliably extract linear acceleration and gyroscope data. The focus was on implementing the `TYPE_LINEAR_ACCELERATION` [?] sensor to filter out gravity, ensuring that only the player’s dynamic force is measured.

Furthermore, a stable TCP socket protocol was established for data transmission. This phase required the sensor sampling rate optimizing to capture rapid stroke movements and implementing a JSON-based serialization scheme.

External Specification

This chapter provides a detailed description of the system from the user’s perspective. It covers the installation requirements, setup procedures, and a walkthrough of typical usage scenarios, including the user interface (UI) and system output.

15 Hardware and Software Requirements

To ensure the correct operation of the “Smart Pool Assistant”, the following minimum requirements must be met.

15.1 Hardware Requirements

• Desktop Workstation (Server):

- CPU: Intel Core i5 (8th Gen) / AMD Ryzen 5 or better.
- RAM: Minimum 8 GB (16 GB recommended for smooth video processing).
- GPU: Dedicated NVIDIA GPU (GTX 1050 or better) recommended for YOLO inference, though CPU-only execution is supported.
- Ports: At least one USB 3.0 port (the phone connection).

• Video Input:

- HD Webcam (720p minimum resolution) mounted on a tripod or ceiling bracket directly above the pool table.

• Mobile Device (Client):

- Smartphone running Android 9.0 (Pie) or higher.
- Must contain hardware sensors: Accelerometer and Gyroscope.
- USB cable for data tethering.

15.2 Software Requirements

- **PC Operating System:** Windows 10/11 or Linux.
- **Python Environment:** Python 3.8 or newer.
- **Android Environment:** Android device with “Developer Options” and “USB Debugging” enabled.
- **Dependencies:** The required Python libraries are listed in the `req.txt` file (e.g., `ultralytics`, `opencv-python`, `inference`, `supervision`).

16 Installation Procedure

The installation process is twofold, involving the desktop server and the mobile client.

16.1 Desktop Installation

1. **Clone the Repository:** Download the project source code from the official GitHub repository. One can download the ZIP file directly:

`https://github.com/ss19190/Smart-9-Ball-Assistant`

Or via terminal: `[language=bash] git clone https://github.com/ss19190/Smart-9-Ball-Assistant`

2. **Install Dependencies:** Open a terminal in the project folder and run: `[language=bash] pip install -r requirements.txt`
3. **Install ADB:** Ensure the Android Debug Bridge (ADB) is installed and added to the system PATH variables.
4. **Configure API Keys:** One needs a valid API Key to access the trained models.
 - Create a free account at `https://roboflow.com`.
 - Navigate to **Settings** → **API** (or your Workspace Settings).
 - Copy your **Private API Key**.

Open a `.env` file in the root directory of the project and paste the key: `API_KEY = your_copied_key_here`

5. **Configure Player Metrics:** To ensure accurate impact force estimations ($F = ma$), the system requires the player’s anthropometric data to calculate the effective arm mass. Open the `config.json` file in the root directory and update the values: `"body_weight_kg": 54, "gender": "female"`
Note: The `gender` field accepts “male” or “female” (case-sensitive). This setting adjusts the arm mass coefficient used in the biomechanical calculations.

16.2 Mobile Installation

1. **Enable Developer Mode:** On the Android phone, go to **Settings** → **About Phone**. Find the "Build Number" entry and tap it 7 times repeatedly until you see a message "You are now a developer!".
2. **Enable USB Debugging:** Go back to the main Settings menu (or System → Advanced), open **Developer Options**, and turn on the switch for **USB Debugging** to ON. Confirm the security prompt if requested.
3. **Install the App:** Locate the **SensorApp.apk** file in the project directory. You can install it manually by copying it to the phone, or simpler, by running this command in your PC terminal (while the phone is connected):
[language=bash] adb install SensorApp.apk Ensure you allow installation from unknown sources if prompted on the device.

17 Activation Procedure

To start a training session, the user must follow a specific sequence of steps to establish the client-server connection.

1. **Connect Hardware:** Plug the webcam into the PC and connect the Android phone via USB.
2. **Setup ADB Forwarding:** This step is important for the phone to communicate with the PC via localhost. Run the following command in the terminal: [language=bash] adb reverse tcp:5555 tcp:5555
3. **Start the Sensor Server:** Run the script responsible for handling telemetry data: [language=bash] python sensors.py
4. **Start the Vision Assistant:** Open a second terminal window and run the AR visualization script: [language=bash] python bilard.py
5. **Connect the App:** Launch the app on the phone and tap the "CONNECT" button. The status should change to green "CONNECTED".

18 Types of Users

The system is designed for a single type of user, although the roles can be conceptually divided:

- **The Player (Trainee):** The primary user who wears the sensor and performs the shots. They interact with the physical game and view the feedback on the screen.
- **The Operator (Optional):** A second person (e.g., a coach) who manages the software, starts/stops the scripts, and analyzes the history logs, though the Player can perform these tasks themselves.

19 User Manual

19.1 Main Interface (Desktop)

The desktop window displays the live camera feed overlaid with augmented reality elements.

- **Ghost Ball Indicator:** A white circle appearing on the aiming line, showing where the cue ball will be at impact.
- **Trajectory Lines:**
 - *Gray Line:* The aiming line extending from the cue stick.
 - *Yellow Line:* The predicted path of the cue ball after impact (Tangent Line).
 - *Green Line:* The predicted path of the object ball.

19.2 Mobile Interface

The mobile app interface is minimal to avoid distraction.

- **Status Bar:** Shows connection state (Disconnected/Connected).
- **Real-time Values:** Displays raw linear acceleration (X, Y, Z) for debugging purposes.
- **Connect Button:** Toggles the TCP connection.

20 System Administration

Since this is a standalone prototype, system administration tasks are limited to:

- **Log Management:** The system generates CSV files (`hit_history.csv`) and image snapshots in the `saved_graphs/` folder. The user should periodically archive or delete old files to save disk space.
- **Calibration:** If the "Ghost Ball" accuracy drifts, the user may need to verify the `BALL_DIAMETER_PX` constant in `bilard.py` to match the current camera height.

21 Security Issues

- **Network Security:** The communication uses unencrypted TCP sockets. This is acceptable for a local USB connection (ADB forwarding), but if ever was implemented and used over public Wi-Fi, the data stream could be intercepted.
- **API Keys:** The Roboflow API key is stored in a `.env` file. This file should not be shared or committed to public version control repositories.

22 Example of Usage

Scenario: Practicing the Cut Shot The player wants to practice a cut shot into the corner pocket.

1. The player sets up the white ball and the target ball (Figure ??).
2. The system detects the balls and waits for the cue stick (Figure ??).
3. The player addresses the cue ball. The system detects the cue pose and renders the “Ghost Ball” next to the target ball.
4. The AR lines show that the current aim will cause the target ball to miss the pocket (hit the rail), as shown in Figure ??.
5. The player adjusts their stance until the Green Line points directly into the pocket (Figure ??).
6. The player executes the stroke.
7. The `sensors.py` script detects a peak acceleration of 33.0 m/s^2 (Figure ??).
8. The system saves a snapshot of the stroke metrics, allowing the player to see if they maintained wrist steadiness during the shot (Figure ??).

23 Working Scenarios

This section illustrates the system in action with screenshots from the prototype, demonstrating the two primary use cases: visual aiming assistance and biomechanical stroke analysis.

Scenario A: Aiming Assistance

Figure ?? demonstrates the vision module in action. The system successfully detects the game state regardless of table size.

- In both subfigures (a) and (b), the system identifies object balls (red bounding boxes) and the cue ball (purple/blue box).
- The white line represents the aiming vector derived from the cue stick.
- The “Ghost Ball” is visualized as a white circle at the predicted point of impact.
- The predicted post-collision paths are rendered as a yellow line (Tangent Line for the cue ball) and a green line (path of the object ball), providing immediate feedback on whether the shot will be successful.

[width=0.5]./graf/usecase.png

Figure 1: Use case diagram.

[width=0.8]./graf/ball-detection/labels-ball.pdf

Figure 2: Analysis of the ball detection dataset: (a) Class distribution showing the massive disproportion between “other” balls and the single “cue ball”; (b) Bounding box density map; (c) Spatial distribution of object centers; (d) Correlation of bounding box width and height.

[width=0.8]./graf/cue-detection/labels-cue.pdf

Figure 3: Analysis of the cue detection dataset: (a) Single-class distribution; (b) Bounding box density map; (c) Spatial distribution of cue centers; (d) Correlation of bounding box width and height.

[b]0.48 [width=]./graf/setup_{ball}.png

Figure 4: Step 1: Initial arrangement of the balls. Final shoot target is the corner pocket.

[b]0.48 [width=]./graf/ball_{detection}.png

Figure 5: Step 2: System detects the object ball (red box) and cue ball (purple box).

Figure 6: Initialization phase.

[b]0.48 [width=]./graf/miss.png

Figure 7: Step 4: Visual feedback indicating a miss (target ball hitting the rail).

[b]0.48 [width=]./graf/good.png

Figure 8: Step 5: Player corrects the aim. The green line now leads into the pocket.

Figure 9: Aiming assistance phase: Real-time correction.

[width=0.85]./graf/app_{shot}.png

Figure 10: Step 7: The system detects the peak acceleration moment corresponding to the impact.

[width=0.85]./graf/screen_{shot}.png

Figure 11: Step 8: Final post-shot analysis saved by the system, showing force (89 N) and stability metrics.

[b]0.8 [width=]./graf/example.png

Figure 12: Visualization on a small table.

[b]0.8 [width=]./graf/example2.png

Figure 13: Visualization on a standard pool table.

Figure 14: Scenario A: The Vision Assistant interface visualizing a valid collision. The system calculates and displays the predicted outcome in real-time.

Scenario B: Stroke Analysis

The second scenario, depicted in Figure ??, illustrates the telemetry part. This sequence shows how the system captures and processes the player's movement.

- Subfigure (a) shows the desktop dashboard in an *Idle* state. The graphs are flat, and the "Max Force" indicator is reset to zero.
- Subfigure (b) captures the state immediately *after* a shot. The system has detected a peak acceleration (22.3 m/s^2) and calculated an impact force of **89 N**. The graphs visualize the stroke profile, allowing the player to check for smoothness.
- The bottom row shows the mobile client acting as the controller. It transitions from a disconnected state (c) to an active streaming state (d).

Internal Specification

This chapter presents the technical details of the solution, including the system architecture, key data structures, and a description of the implementation of the most critical modules in Python and Java. It outlines the data flow and communication protocols established between the mobile telemetry client and the desktop vision server. It also covers the algorithmic logic behind the "Ghost Ball" trajectory prediction and the physics-based impact force estimation.

24 System Concept

The system concept is based on the fusion of sensory and visual data to assist in playing billiards. The system consists of:

1. **Vision Module (AI):** It analyzes the camera feed, detects billiard balls and the cue, and subsequently calculates the "Ghost Ball" — the predicted position of the cue ball at the moment of impact.
2. **Telemetry Module:** It analyzes the force of the impact and the smoothness of the player's movement based on IMU sensor data (accelerometer, gyroscope) transmitted via an USB cable from a smartphone.

25 System Architecture

The system operates in a client-server architecture, as illustrated in Figure ?? . The smartphone (Client) collects data and transmits it via TCP to the workstation (Server), which processes the camera image in parallel to the transmission.

The main unit (PC) runs two independent processes (Python scripts):

- **Vision Process (bilard.py):** It utilizes the `inference` and `supervision` libraries for object detection and vector calculations.
- **Sensor Server (sensors.py):** A multi-threaded TCP server that receives JSON data, parses it, and visualizes graphs using the `OpenCV` library.

26 Description of Data Structures

26.1 Network Payload (JSON)

Communication between the Android device and the PC is handled using the JSON format. A main feature is the transmission of linear acceleration (gravity excluded) and angular velocity, organized as shown in Figure ?? .

26.2 Physics Vectors (Python)

The vision module operates on vectors from the `NumPy` library to compute trajectories. Figure ?? illustrates how the cue ball position and the aiming vector are initialized within the Python script.

27 Components and Modules

27.1 Mobile Module (Android)

The mobile application was developed in Java. The key component is the `SensorManager`. The `TYPE_LINEAR_ACCELERATION` sensor was utilized instead of the standard accelerometer to eliminate the influence of gravity on the impact force measurement, as demonstrated in Figure ?? .

27.2 Vision Module (Computer Vision)

This module utilizes two separate neural network models hosted on the Roboflow platform, optimized for instant inference:

- **Ball Detection:** A generic object detection model (version `ball-detection-bzirz/3`) based on the state-of-the-art **YOLOv12** [?] architecture. It is trained to localize billiard balls and classifies objects into two distinct classes: `cue ball` and `other`. The model works on a standard input resolution (640×640) and outputs bounding box coordinates. YOLOv12 was

[b]0.48 [width=]./graf/sensorExample.png

Figure 15: Desktop interface in a state before a stroke.

[b]0.48 [width=]./graf/strokeExample.png

Figure 16: Desktop interface showing impact analysis showing max force of 89 N.

[b]0.48 [height=5.5cm]./graf/phone1.png

Figure 17: Mobile app in disconnected state.

[b]0.48 [height=5.5cm]./graf/phone2.png

Figure 18: Mobile app connected, streaming sensor data.

Figure 19: Scenario B: Stroke Analysis workflow. The top row shows the PC application analyzing the stroke data, where (a) is pre-hit and (b) is post-hit showing impact metrics. The bottom row shows two possible states of the mobile sensor app (c, d).

[width=0.9]./graf/arch.png

Figure 20: System Architecture Diagram and logic flow.

```
[ language=Java, basicstyle=, breaklines=true, captionpos=b ] // JSON
format generated in MainActivity.java String json =
String.format(Locale.US, "acc_x'%gyro_x'ax, ay, az, gx, gy, gz);
```

Figure 21: JSON structure creation in Java (Android).

```
[ language=Python, basicstyle=, breaklines=true, captionpos=b ] Position
representation in bilard.py cue_ball_center = np.array([x, y], dtype =
np.float32)aim_vector = normalize_vector(tip_pos - handle_pos)
```

Figure 22: NumPy vector initialization for cue ball and aiming direction.

```
[ language=Java, basicstyle=, breaklines=true, captionpos=b ] //
MainActivity.java - sensor initialization linearAcceleration =
sensorManager.getDefaultSensor( Sensor.TYPE_LINEAR_ACCELERATION);
```

Figure 23: Initialization of the Linear Acceleration sensor in Android.

chosen over its predecessors for its superior extraction capabilities and bounding box stability, which are critical for maintaining a steady line.

- **Cue Detection:** A pose estimation model (version `cue-detection-ciazj/3`) utilizing the **YOLOv8-pose** [?] architecture. Unlike the ball detector, this model outputs semantic keypoints representing the **tip** and **handle**, necessary for calculating the aiming vector. YOLOv8-pose was selected for its lightweight architecture and inference speed. Studies indicate that while established frameworks like OpenPose provide valuable insights, their high computational complexity makes it harder for real-time integration [?]. In contrast, YOLOv8-based architectures have been demonstrated to outperform such models in efficiency, achieving higher Object Keypoint Similarity (OKS) scores (e.g., 76.1% vs 75.2% for OpenPose) while maintaining the low latency ($< 30\text{ ms}$) required for a smooth Augmented Reality experience.

28 Overview of Key Algorithms

28.1 Ghost Ball Algorithm

The implementation translates the vector mathematics and geometric rules defined in Section 2.3.1 into Python code using the NumPy library. The function `find_ghost_ball_position` calculates the projection vector and validates the perpendicular distance to render the visu-

28.2 Impact Force Calculation (Physics)

The `sensors.py` module implements the force estimation model derived in Section 2.3.2. To ensure flexibility, the system does not use hard-coded anthropometric constants in the source code. Instead, it dynamically loads the user's attributes (weight and gender) from a `config.json` file.

Based on these attributes, the system selects the appropriate effective mass coefficient (μ_{gender}) — as detailed in the theoretical analysis (see Section 2.3.2) — and computes the personalized arm mass (m_{arm}). The final impact force is then derived by applying the calculated mass to the linear acceleration vector received from the mobile device.

Additionally, the algorithm implements a peak detection logic (refer to Section 2.4.2) that specifically identifies the *second* significant peak in the signal buffer. This implementation detail allows the system to distinguish the actual forward strike from the preparatory backswing movement.

29 Implementation Details

29.1 Thread Synchronization (Python)

The TCP server in `sensors.py` operates in a separate thread (`daemon=True`) to avoid blocking the interface drawing loop (GUI) in the OpenCV library, if one

would like to run both modules at the same time, as illustrated in Figure ??.

29.2 Network Handling (Android)

In the Android application, a separate thread was employed for network operations to avoid the `NetworkOnMainThreadException`. Data is transmitted at a frequency of approximately 100 *Hz* (every 10 *ms*), ensuring smooth physics representation.

30 Applied Design Patterns

- **Listener Pattern (Android):** The implementation of the `SensorEventListener` interface enables reactive handling of sensor value changes only when they occur [?].
- **Producer-Consumer:** The TCP thread (Producer) receives data and updates the global `sensor_data` structure, which is subsequently consumed by the main visualization loop [?].

Verification and Validation

This chapter describes the testing methods adopted to maintain the consistency and accuracy of the developed system. It covers the testing paradigm, the scope of test cases, a detailed analysis of encountered software defects, and the final experimental results.

31 Testing Paradigm

To ensure an organized approach to verification, the **V-Model** (Verification and Validation Model) was adopted. This model focuses on the relationship between the design phase and the testing phase. The testing process was divided into three distinct levels:

1. **Unit Testing:** Individual components were tested in isolation. For the Python server, this involved testing the vector calculation functions (e.g., `find_hostball_position`) and the impact force formula ($F=ma$) with known static values to ensure mathematical correctness.
2. **System Testing:** The full system was validated in a real-world environment. This involved a player performing actual shots while the system tracked the game state and visualized the analytics in real-time.

32 Testing Scope and Test Cases

The testing scope covered both the functional requirements (correct physics calculations, correct object detection) and non-functional requirements (performance, latency). Table ?? summarizes the key test scenarios.

Table 1: Summary of Key Test Cases.

ID	Test Scenario	Expected Result	Status
TC-01	JSON Packet Transmission	Server receives valid JSON structure without corruption.	Passed
TC-02	Ghost Ball Projection	Projector line intersects the target ball center.	Passed
TC-03	High-Velocity Shot Detection	Accelerometer detects peak $> 20 \text{ m/s}^2$.	Passed
TC-04	Occlusion Handling	System maintains ID of balls when partially obscured.	Partial

The status “Partial” for TC-04 means that the tracking algorithm can handle minor occlusions (e.g., when balls momentarily touch). However, during significant occlusion — for example when a ball is covered by the player’s hand — the detection logic may fail, as illustrated in Figure ??.

33 Detected and Fixed Bugs

During the implementation and testing phases, multiple significant software defects were identified. Below is a description of the most significant bugs and the solutions applied.

33.1 Android UI Thread Blocking (UI Lag)

Problem: The initial version of the Android application exhibited severe interface lag and unresponsiveness during data recording. The accelerometer sensor was configured to `SENSOR_DELAY_FASTEST`, generating events at approximately 100 Hz . The application attempted to update the on-screen `TextView` elements inside every sensor callback. Since the UI rendering thread operates at a lower frequency (approx. 60 Hz), the event queue became overwhelmed, causing the displayed values to “trail” behind reality (e.g., the value would continue rising long after the phone had stopped moving).

Solution: The implementation was modified to decouple data collection from UI updates. A throttling mechanism was introduced to update the UI elements only once every 100 ms (10Hz), while the data transmission to the server continued at the full 100 Hz rate using a background thread.

33.2 TCP Stream Fragmentation

Problem: Occasionally, the Python server would crash with a `JSONDecodeError`. This occurred because TCP is a stream-oriented protocol, not a message-oriented one. At high transmission rates, multiple JSON objects were merged into a single buffer read, or a single JSON object was split across two reads.

Solution: A delimiter (newline character `\n`) was appended to every message sent from Android. On the server side, a buffer was implemented to accumulate incoming bytes until a full delimiter was found, making sure that only complete JSON strings were passed to the parser.

33.3 Ghost Ball Jitter

Problem: The detected position of the cue ball and cue tip fluctuated slightly (by 1-2 pixels) due to noise in the camera, even when the objects were stationary. This caused the projected “Ghost Ball” vector to shake rapidly, making it difficult for the user to aim.

Solution: A moving average filter (size $N = 5$) was applied to the coordinates of the detected objects. This smoothed out the high-rate noise without losing acceptable responsiveness to intentional movements.

34 Computer Vision Model Evaluation

To ensure the reliability of the system, the trained neural networks were subjected to rigorous assessment using a hold-out test set methodology [?].

According to Bishop, this approach involves partitioning the available data into disjoint subsets: a training set, used to determine the model coefficients (weights), and a separate hold-out set, used to optimize model complexity and evaluate generalization error.

34.1 Ball Detection Model Results

The Ball Detection model, built on the YOLOv12 architecture, showed outstanding performance, obtaining a **mean Average Precision (mAP@0.5)** of **0.991**.

The mAP metric acts as the mean of the Average Precision (AP) calculated across all C classes. The AP itself represents the area under the Precision-Recall curve $p(r)$:

$$mAP = \frac{1}{|C|} \sum_{i=1}^{|C|} AP_i = \frac{1}{|C|} \sum_{i=1}^{|C|} \int_0^1 p_i(r) dr \quad (11)$$

where Precision (P) and Recall (R) are derived from True Positives (TP), False Positives (FP), and False Negatives (FN) calculated at a fixed Intersection over Union (IoU) threshold (specifically $\text{IoU} \geq 0.5$ for mAP@0.5):

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN} \quad (12)$$

This result indicates nearly perfect detection abilities at the standard threshold (Figure ??a).

- **Training Performance:** As illustrated in the training plots (see Figure ??), the box loss and classification loss converged rapidly within the first 50 epochs, stabilizing with no signs of overfitting. The precision and recall metrics consistently improved, plateauing near 1.0.
- **F1-Score and Threshold Selection:** The optimal confidence threshold was determined based on the F1-Confidence curve generated from the validation set (Figure ??b). To follow the hold-out methodology and avoid data leakage, the threshold was determined using only the validation data. Once selected, it was fixed and then used unchanged in the production code.
- **Classification Performance:** The confusion matrix (Figure ??) confirms the model's accuracy on the test set. The model correctly identified **110** Cue Ball instances and **1043** Object Balls ("other"). Misclassifications were negligible, with only minimal instances of background falsely detected as balls (False Positives).

34.2 Cue Detection Model Results

The Cue Detection model, utilizing pose estimation (YOLOv8-pose), was evaluated based on both bounding box detection and, more importantly, the accuracy of keypoint localization (Object Keypoint Similarity). Accurate keypoint detection is critical for determining the aiming vector of the cue stick.

- **Training Performance:** Similar to the ball detection model, the training metrics (Figure ??) demonstrated stable convergence. The box loss and pose loss decreased steadily over the epochs, while precision and recall metrics improved consistently, indicating that the model effectively learned the spatial features of the cue stick without significant overfitting.
- **Pose Estimation Accuracy:** The model reached a **Pose mAP@0.5** of **0.954** (Figure ??a), indicating high precision in locating the cue tip and handle keypoints.
- **F1-Score Analysis:** The F1-Confidence curve for pose estimation (Figure ??b) identifies an optimal confidence threshold of **0.48**, where the model obtains a peak F1 score of **0.93**.
- **Classification Performance:** The confusion matrix (Figure ??) displays the model's ability to distinguish the cue from the table background. Out of 76 actual cue instances in the validation set, the model correctly identified 70, with only a minor number of background false positives.

```
[ language=Python, basicstyle=, breaklines=true, captionpos=b ] def
    find_ghost_ball_position(cue_ball_pos, aim_vector, target_ball_pos): vec_to_target =
        target_ball_pos - cue_ball_pos
    projection_length = np.dot(vec_to_target, aim_vector)
    Calculating the closest point on the shot line
    closest_point = cue_ball_pos + aim_vector * projection_length
    perp_dist = np.linalg.norm(target_ball_pos - closest_point)
Correction for ball radius (backward offset) if perp_dist < BALL_DIAMETER_PX :
    back_offset = math.sqrt(BALL_DIAMETER_PX ** 2 - perp_dist ** 2)
    dist_impact = projection_length - back_offset
    return cue_ball_pos + aim_vector * dist_impact
```

Figure 24: Vector-based Ghost Ball calculation implementation.

```
[ language=Python, basicstyle=, breaklines=true, captionpos=b ]
sensors.py - Threading implementation
threading.Thread(target=tcp_server_thread, daemon = True).start()
while True:    Main GUI Loop (OpenCV) cv2.imshow("Hit Analysis", window)
    if cv2.waitKey(20) & 0xFF == ord('q'): break
```

Figure 25: Implementation of the separate thread for TCP communication.

[width=0.85]./graf/partial.png

Figure 26: Visualization of the occlusion limitation (Test Case TC-04). The system successfully detects visible balls, but fails to identify the orange ball (top right) which is partially covered by the player's hand.

[width=1.0]./graf/ball-detection/results.pdf

Figure 27: Training performance metrics over 150 epochs. The graphs display the convergence of Box and Classification losses (left) and the improvement of Precision, Recall, and mAP metrics (right).

[width=1.0]./graf/ball-detection/charts-ball.pdf

Figure 28: Performance metrics for the Ball Detection model: a) Precision-Recall Curve showing mAP@0.5 of 0.985; b) F1-Confidence Curve indicating the optimal confidence threshold of 0.59.

[width=0.8]./graf/ball-detection/confusion_matrix.pdf

Figure 29: Confusion Matrix demonstrating raw detection counts for Cue Ball, Other, and Background classes.

[width=1.0]./graf/cue-detection/results.pdf

Figure 30: Training performance metrics for the Cue Detection model. The graphs illustrate the convergence of Box and Pose losses, confirming stable learning.

[width=1.0]./graf/cue-detection/charts-cue.pdf

Figure 31: Performance metrics for the Cue Detection model: a) Precision-Recall Curve for Pose Estimation (mAP@0.5 of 0.954); b) F1-Confidence Curve for the cue stick keypoints (peak at 0.93).

35 System Validation

To validate the real-word applicability of the system, an experiment was conducted to verify the consistency and distinctiveness of the impact force estimation algorithm. The test was performed by a female subject with a body weight of 54 kg. Based on the anthropometric configuration described in Chapter 5, the system calculated the effective arm mass as:

$$m_{arm} = 54kg \times 4.97\% \approx 2.68kg$$

35.1 Force Estimation Consistency

The test procedure involved performing a series of shots categorized by subjective intensity. Initially, three categories were planned: Soft, Medium, and Hard.

However, during the experiment, it was observed that shots categorized as “Soft” (gentle taps for exact positioning) consistently generated acceleration values below the system’s triggering threshold (20 m/s^2). Consequently, these shots were filtered out by the noise reduction algorithm.

This distinct behavior is visually demonstrated in Figure ?? . As shown, the “Soft” tap did not breach the threshold, resulting in the system correctly ignoring the event.

In contrast, “Medium” and “Hard” shots successfully triggered the recording logic, capturing distinct force profiles. The Medium shot, presented in Figure ?? , shows a clear peak typical for standard gameplay. Figure ?? illustrates a Hard shot, characterized by high intensity and a much larger force value.

This thresholding method corresponds to methodologies found in kinematic research, where velocity or acceleration thresholds are fundamental for defining discrete movement segments and assessing smoothness [?]. This confirms that the threshold effectively separates between active gameplay strikes and minor refinements or accidental movements.

Therefore, the assessment was targeted on the two distinct categories ($N = 20$ total):

1. **Medium shots:** Standard shots used during regular gameplay.
2. **Hard shots:** Break-type shots or power shots.

Table ?? presents the raw force values recorded for each attempt.

Table 2: Raw recorded force values (in Newtons) for the experimental session ($m_{arm} = 2.68$ kg).

Attempt #	Medium Shot (N)	Hard Shot (N)
1	63.4	109.7
2	65.7	115.4
3	74.4	107.4
4	83.1	126.6
5	54.9	114.9
6	56.5	128.0
7	76.9	124.9
8	68.9	110.2
9	63.2	92.8
10	54.0	96.7

Based on the raw data, a statistical analysis was performed to calculate the Mean Force (F_{avg}) and Standard Deviation (σ) for each category [?]. The results are summarized in Table ??.

Table 3: Statistical analysis of impact force measurements ($N = 20$).

Shot Category	Min (N)	Max (N)	Mean $\pm$ Std Dev (N)
Medium Shot	54.0	83.1	66.1 $\pm$ 9.3
Hard Shot	92.8	128.0	112.66 $\pm$ 11.33

35.2 System Latency and Performance

A secondary test was conducted to evaluate the real-time capabilities of the system. The total latency was measured as the time difference between the sensor event timestamp (on Android) and the visualization update on the PC.

- **Network Latency:** Average 15 *ms* (local Wi-Fi network).
- **Processing Time:** Average 25 *ms* (Vision Module inference per frame).
- **Frame Rate:** The vision module maintained a stable 30 FPS on the test workstation.

These efficiency measures confirm that the system can provide a fluid visual feedback. Thanks to this user can relate the data to the just-performed action.

Conclusions

The main goal of this thesis was to design and implement the "Smart 9-Ball Assistant", a hybrid system capable of augmenting the game of billiards with real-time visual guidance and biomechanical feedback. The resulting prototype successfully integrates Computer Vision and Telemetry data to bridge the gap between physical execution and geometric visualization.

36 Achievement of Objectives

The project goals, as defined in the introduction, have been met with the following outcomes:

- **Computer Vision Module:** An effective detection pipeline was developed using a custom-trained **YOLOv12** model for billiard balls and **YOLOv8-pose** for the cue stick. The validation results confirmed exceptional accuracy, with the ball detection model attaining a mean Average Precision (mAP@0.5) of **0.991**, making sure that the system can reliably track the game state under different lighting conditions.
- **Trajectory Prediction:** The "Ghost Ball" algorithm was successfully implemented. By calculating the intersection of the cue stick vector with the target ball's geometry, the system provides immediate visual feedback (Augmented Reality overlays) that helps users visualize the aim line and the tangent line.
- **Telemetry and Biomechanics:** The mobile sensor module, developed for the Android platform, reliably obtains high-frequency inertial data. By implementing the physics-based force model ($F = ma$) and utilizing anthropometric data (Plagenhoef et al.[?]), the system can estimate the impact force in Newtons. Experimental results confirmed that the system allows for the objective differentiation of shot intensity (e.g., distinguishing "Medium" vs. "Hard" shots with clear statistical separation).
- **Real-time Performance:** The system satisfies the non-functional requirement for low latency. The vision pipeline operates at approximately 30 FPS, and the network transmission delay (approx. 15 ms) is negligible for the purpose of static aiming adjustment.

37 Difficulties Experienced

During the development lifecycle, multiple technical challenges were encountered and addressed:

- **Network Fragmentation:** A significant issue arose with the TCP communication where high-frequency JSON packets were being coalesced or split, causing parsing errors. This was resolved by implementing a stream buffering mechanism with strict delimiter handling.
- **Android UI Performance:** The initial mobile application suffered from severe "UI Lag" due to processing sensor events on the main thread. Decoupling the data transmission thread from the UI rendering thread (via the producer-consumer pattern) was necessary to maintain stable performance.

- **Visual Jitter:** Raw detection data from the camera resulted in “shaking” projection lines, which made aiming difficult. A temporal smoothing algorithm (Moving Average Filter) was implemented to stabilize the vector calculations without introducing perceptible lag.
- **Dataset Imbalance:** Training the neural network required addressing the significant class imbalance between the single “cue ball” and the fifteen “object balls.” This was mitigated via focused data augmentation during the pre-processing phase.

38 Future Development

While the current prototype is fully functional, several paths for additional development have been identified to enhance the system’s commercial viability and user experience:

- **Perspective Correction (Homography):** Currently, the system requires a strictly top-down camera view. Future iterations could implement a homography transformation matrix to allow the camera to be placed at an angle (e.g., on a tripod at the side of the table), making the setup more flexible.
- **Wireless Sensor Integration:** Replacing the USB tethering with Bluetooth Low Energy (BLE) or Wi-Fi Direct would significantly improve player comfort, removing the physical cable constraint during the stroke.
- **Advanced Physics Engine:** The current model assumes elastic collisions. Further development might incorporate the effects of “English” (spin) and cloth friction into the trajectory prediction to simulate curve shots and bank shots more accurately.
- **Gamification:** The system could be expanded with a “Training Mode” that creates a challenge for the player to hit specific force targets or execute pre-defined shots, tracking their progress over time in a cloud-based database.

Index of abbreviations and symbols

Abbreviations

ADB Android Debug Bridge

AI Artificial Intelligence

API Application Programming Interface

AR Augmented Reality

CNN Convolutional Neural Network
 CV Computer Vision
 FPS Frames Per Second
 IMU Inertial Measurement Unit
 IoU Intersection over Union
 JSON JavaScript Object Notation
 mAP mean Average Precision
 TCP Transmission Control Protocol
 UI User Interface
 USB Universal Serial Bus
 YOLO You Only Look Once

Symbols

F impact force [N]
 m_{arm} effective mass of the arm [kg]
 μ_{gender} anthropometric arm mass coefficient [%]
 $\vec{a}$ linear acceleration vector
 $\vec{v}_{aim}$ normalized aiming direction vector
 P_{cue} center position of the cue ball
 P_{ghost} calculated position of the ghost ball
 L_{proj} length of the projection vector
 $d_{\perp}$ perpendicular distance to the aiming line

Listings

This chapter includes the complete source code for the configuration file and the two main backend modules (Vision and Telemetry) developed for the system.

39. Configuration File (config.json)

The user-specific configuration file utilized to define anthropometric data for the physics engine.

[language=Java, caption=Content of config.json, label=lst:config_{json}]”*body_weight_kg*” : 54,”*gender*” : ”

40. Vision Module (bilard.py)

The main computer vision script responsible for object detection (YOLO), cue keypoint tracking, and the "Ghost Ball" trajectory calculation.

```
[language=Python, basicstyle=, breaklines=true, caption=Vision Module implementation,
label=lst:bilard_py]import cv2
import supervision as sv
from inference import get_model
import numpy as np
import math
from dotenv import load_dotenv()
-- CONFIGURATION --
API_KEY = os.getenv("API_KEY")
DETECTION_MODEL = "ball-detection-bzirz/3"
KEYPOINTS_MODEL = "cue-detection-ciazj/3"
BALL_DIAMETER_PX = 45
def normalize_vector(v):
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm
def find_ghost_ball_position(cue_ball_pos, aim_vector, target_ball_pos):
    vec_o_t_target = target_ball_pos - cue_ball_pos
    projection_length = np.dot(vec_o_t_target, aim_vector)
    if projection_length <= 0:
        return None, float('inf')
    closest_point_on_line = cue_ball_pos + aim_vector * projection_length
    perpendicular_dist = np.linalg.norm(target_ball_pos - closest_point_on_line)
    if perpendicular_dist < BALL_DIAMETER_PX:
        back_offset = math.sqrt(BALL_DIAMETER_PX * 2 - perpendicular_dist * 2)
        distance_o_t_impact = projection_length - back_offset
        ghost_ball_pos = cue_ball_pos + aim_vector * distance_o_t_impact
        return ghost_ball_pos, distance_o_t_impact
    return None, float('inf')
def main():
    model_obj = get_model(model_id = DETECTION_MODEL, api_key = API_KEY)
    model_kp = get_model(model_id = KEYPOINTS_MODEL, api_key = API_KEY)
    cap = cv2.VideoCapture(0)
    box_annotator = sv.BoxAnnotator(thickness=2)
    print("Start! Press 'q' to exit.")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        annotated_frame = frame.copy()
        1. OBJECT DETECTION
        results_obj = model_obj.infer(frame)[0].detections
        annotated_frame = box_annotator.annotate(scene=annotated_frame, detections=detections)
        cue_ball_center = None
        other_ball_centers = []
        for i in range(len(detections)):
            class_name = detections.data['class_name'][i]
            box = detections.xyxy[i]
            center_x = int((box[0] + box[2]) / 2)
            center_y = int((box[1] + box[3]) / 2)
            center_point = np.array([center_x, center_y], dtype=np.float32)
            if class_name == "cueball" or class_name == "cue-ball":
                cue_ball_center = center_point
            elif class_name == "other":
                other_name = f"coloredball_{other_ball_centers.append(center_point)}"
        2. CUE KEYPOINT DETECTION
        results_kp = model_kp.infer(frame)[0]
        tip_pos = None
        handle_pos = None
        if hasattr(results_kp, "predictions"):
            for pred in results_kp.predictions:
                if hasattr(pred, "keypoints"):
                    for kp in pred.keypoints:
                        if kp.confidence > 0.4:
                            pos = np.array([kp.x, kp.y], dtype=np.float32)
                            if kp.class_name == "tip":
                                tip_pos = pos
                            elif kp.class_name == "handle":
                                handle_pos = pos
        3. DRAWING LINES AND PREDICTION
        if cue_ball_center is not None and tip_pos is not None and handle_pos is not None:
            -- DRAWING CUE LINE (Handle -> Tip) --
            handle_int = tuple(handle_pos.astype(int))
            tip_int = tuple(tip_pos.astype(int))
            thick_line = cv2.line(annotated_frame, handle_int, tip_int, (255, 0, 0), 4)
            Calculate aiming vector based on the cue
            aim_vector = tip_pos - handle_pos
            aim_dir = normalize_vector(aim_vector)
            Looking for collision
            closest_ghost_ball = None
            closest_target_ball = None
            min_distance = float('inf')
            for target_ball in other_ball_centers:
                ghost_pos, distance = find_ghost_ball_position(cue_ball_center, aim_dir, target_ball)
                if ghost_pos is not None and distance < min_distance:
                    min_distance = distance
                    closest_ghost_ball = ghost_pos
                    closest_target_ball = target_ball
            -- DRAWING PREDICTION LINES --
            cue_start_int = tuple(cue_ball_center.astype(int))
            Line connecting Tip to Cue Ball (shows "aiming")
            cv2.line(annotated_frame, tip_int, cue_start_int, (100, 100, 100), 2, cv2.LINE_AA)
            if closest_ghost_ball is not None:
                ghost_int = tuple(closest_ghost_ball.astype(int))
                target_int = tuple(closest_target_ball.astype(int))
                1. Line Cue Ball -> Ghost Ball (continuation of cue line)
                cv2.line(annotated_frame, cue_start_int, ghost_int, (255, 255, 255), 2, cv2.LINE_AA)
                Circle at ghost ball location
                cv2.circle(annotated_frame, ghost_int, int(BALL_DIAMETER_PX / 2), (255, 255, 255), 1)
                Rebound physics
                impact_vector = closest_target_ball - closest_ghost_ball
                impact_dir = normalize_vector(impact_vector)
                tangent_dir = np.array([-impact_dir[1], impact_dir[0]])
                if np.dot(tangent_dir, aim_dir) < 0:
                    tangent_dir = -tangent_dir
                2. Target Ball Path (Green)
                target_end_pos = closest_target_ball + impact_dir * 200
                cv2.arrowedLine(annotated_frame, target_int, target_end_pos, (0, 255, 0), 2)
                3. Cue Ball Path (Yellow)
                cue_end_pos = closest_ghost_ball + tangent_dir * 200
                cv2.arrowedLine(annotated_frame, cue_start_int, cue_end_pos, (0, 255, 0), 2)
            else:
                If no collision, draw line to infinity from the cue ball
                infinity_point = cue_ball_center + aim_dir * 1000
                cv2.line(annotated_frame, cue_start_int, tuple(infinity_point.astype(int)), (200, 200, 200), 1, cv2.LINE_AA)
            cv2.imshow("Billiards AI - Full Path", annotated_frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break
```



```
cap.release() cv2.destroyAllWindows()
if name=="__main__":
```

41. Telemetry Module (sensors.py)

The backend server that handles TCP communication with the mobile device, calculates impact force ($F = ma$) using anthropometric data, and visualizes the results on a dashboard.

```
[language=Python, basicstyle=, breaklines=true, caption=Telemetry Module implementation,
label=lst:sensors_py]import cv2import numpy as npimport mathimport socketimport jsonimport threadingimport timeimport csv
-- CONFIGURATION -- PORT = 5555 HISTORY_SIZE = 200CONFIGFILE = "config.json"
Files and Folders CSVFILE = "hit_history.csv"IMAGEFOLDER = "saved_graphs"
Delays DELAY_SNAPSHOT1 = 0.8DELAY_SNAPSHOT2 = 3.0
-- DATA FROM PLAGENHOEF ET AL. (1983) -- Sum of Hand + Forearm + Upper Arm percentages Source:
Table 4, Segment Weights as Percentages of Total Body Weight ARM_PERCENTAGE_MALE = 0.05770.65ARM_PERCENTAGE_FEMALE = 0.04970.5
def load_armmass(): """Calculates arm mass based on Plagenhoeft et al. (1983) data.""" default_armmass = 4.0
if not os.path.exists(CONFIGFILE): print(f"Config file not found. Using default arm mass :
default_armmasskg") return default_armmass
try: with open(CONFIGFILE, 'r') as f: config = json.load(f)
weight = float(config.get("body_weight_kg", 75.0)) gender = config.get("gender", "male").lower()
Calculate biological arm mass based on research percentages if gender == "female": bio_armmass =
weight * ARM_PERCENTAGE_FEMALE print(f"Loaded settings for FEMALE : weightkg ->
Armmass : bio_armmass : .2fkg(4.97) else : bio_armmass = weight * ARM_PERCENTAGE_MALE print(f"Loaded settings for MALE :
weightkg -> Armmass : bio_armmass : .2fkg(5.77)
total_armmass = bio_armmass print(f"Total Effective Mass (Bio) : total_armmass : .2fkg") return total_armmass
except Exception as e: print(f"Error reading config: {e}. Using default: default_armmasskg") return default_armmass
PHYSICAL CONSTANTS - CALCULATED DYNAMICALLY ESTIMATED ARMMASS = load_armmass()
Thresholds THRESHOLD_HIT = 20.0m/s^2 (Hit detection threshold) THRESHOLD_PEAK = 15.0m/s^2 (Threshold to recognize a peak)
-- GLOBAL VARIABLES -- sensor_data = {"acc_x": 0.0, "acc_y": 0.0, "acc_z": 0.0, "gyro_x": 0.0, "gyro_y": 0.0, "gyro_z": 0.0, "last_update": 0.0}
Session Peaks peak_acc_current = 0.0m/s^2 peak_force_current = 0.0N (Newtons) peak_gyro_current = 0.0deg/s
History Deques history_acc = deque([0]*HISTORY_SIZE, maxlen=HISTORY_SIZE) Acceleration history_force_n = deque([0]*HISTORY_SIZE, maxlen=HISTORY_SIZE) Force(N) history_gyro = deque([0]*HISTORY_SIZE, maxlen=HISTORY_SIZE) Rotation
shot_trigger_time = 0 waiting_for_snapshot1 = False waiting_for_snapshot2 = False
Variables to save (Values from impact moment) saved_acc = 0.0 saved_force_n = 0.0 saved_gyro = 0.0
Function to save file, updated with Newtons def save_file(val_acc, val_force_n, val_gyro, image_snapshot):
if not os.path.exists(IMAGEFOLDER): os.makedirs(IMAGEFOLDER)
now = datetime.now() timestamp = now.strftime(" Save only one type of image (MOMENT) png filename =
f"IMAGEFOLDER/hit_{timestamp}_MOMENT.png"
Background for text cv2.rectangle(image_snapshot, (0, 550), (800, 600), (0, 0, 0), -1)
desc_mode = " (at impact)"
INFO ON IMAGE (Added Newtons) info_text = f"ACC : {val_acc : .1f}m/s^2 | FORCE : {val_force_n : .0f}N | ROTATION :
val_gyro : .0fdeg/s" cv2.putText(image_snapshot, info_text, (20, 580), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
cv2.putText(image_snapshot, f"HIT ANALYSIS {desc_mode}", (20, 550), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.imwrite(png_filename, image_snapshot)
SAVE TO CSV (Added Force_N column) file_exists = os.path.isfile(CSVFILE) with open(CSVFILE, mode='a', newline='') as f: writer = csv.writer(f, delimiter=',') if not file_exists: writer.writerow(["Date", "Time", "Acc_m_s2", "a'", "newline"])
date_str = now.strftime("time_str = now.strftime("writer.writerow([date_str, time_str, f"{val_acc : .2f}", f"{val_force_n : .1f}", f"{val_gyro : .0f}deg/s", f"SAVED: Acc: {val_acc : .1f}, Force: {val_force_n : .0f}N, Gyro: {val_gyro : .0f}deg/s")
print(f"SAVED: Acc: {val_acc : .1f}, Force: {val_force_n : .0f}N, Gyro: {val_gyro : .0f}deg/s")
def tcp_server_thread(): global sensor_data server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server_socket.bind(('0.0.0.0', PORT)) server_socket.listen(1) print(f"Server listening on port {PORT}...")
while True: conn, addr = server_socket.accept() with conn: buffer = "" last_packet_time = time.time()
while True: try: data_chunk = conn.recv(4096) if not data_chunk: break buffer += data_chunk.decode('utf-8') if len(buffer) > 8192: buffer = buffer[-4096:]
while "" in buffer: line, buffer = buffer.split("", 1) if not line.strip(): continue try:
js = json.loads(line) ax, ay, az = float(js.get("acc_x", 0)), float(js.get("acc_y", 0)), float(js.get("acc_z", 0)) gx, gy, gz = float(js.get("gyro_x", 0)), float(js.get("gyro_y", 0)), float(js.get("gyro_z", 0))
```

```

gx, gy, gz = gx * 57.2958, gy * 57.2958, gz * 57.2958
now = time.time() last_packet_time = now
1. Total Acceleration (a) total_acc = math.sqrt(ax**2 + ay**2 + az**2)
2. Force (F = m*a) force_n = total_acc * ESTIMATED_ARM_ASS
3. Total Rotation total_gyro = math.sqrt(gx**2 + gy**2 + gz**2)
sensor_data["acc_x"] = ax; sensor_data["acc_y"] = ay; sensor_data["acc_z"] = az; sensor_data["gyro_x"] =
gx; sensor_data["gyro_y"] = gy; sensor_data["gyro_z"] = gz; sensor_data["last_update"] = now
history_acc.append(total_acc); history_force_n.append(force_n); Addtohistoryhistory_gyro.append(total_gyro)
except json.JSONDecodeError: pass except Exception: break except Exception as e: print(f"Server
error: {e}")
def draw_graph(img, data, x_start, y_start, width, height, color, scale_factor = 2.0, title = "", unit =
"", grid_step = 10): cv2.rectangle(img, (x_start, y_start), (x_start+width, y_start+height), (30, 30, 30), -1); cv2.rectangle(img, (x_start
width, y_start+height), (100, 100, 100), 1); base_line_y = y_start+height-10
max_val_visible = int(height/scale_factor) for val in range(0, max_val_visible, grid_step): y_pos =
base_line_y-int(val*scale_factor) if y_pos < y_start: break if val > 0: cv2.line(img, (x_start, y_pos), (x_start+
width, y_pos), (60, 60, 60), 1); cv2.putText(img, str(val), (x_start+5, y_pos-2), cv2.FONT_HERSHEY_SIMPLEX, 0.35, (150, 150, 150),
points = [] if len(data) > 0: step = width / len(data) for i, value in enumerate(data): x
= x_start+int(i*step); y = base_line_y-int(value*scale_factor); y = max(y_start, min(y, base_line_y)); points.append((x, y))
if len(points) > 1: cv2.polylines(img, [np.array(points)], isClosed=False, color=color, thickness=2)
cv2.putText(img, title, (x_start+5, y_start+20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (220, 220, 220), 1) if data:
cv2.putText(img, f"data[-1]: {data[-1]:.1f} unit", (x_start+width-120, y_start+20), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
def main(): global shot_rigger_time, waiting_for_snapshot1, waiting_for_snapshot2; global saved_acc, saved_force_n, saved_gyro; global
threading.Thread(target=tcp_server_thread, daemon=True).start()
cv2.namedWindow("Hit Analysis") cv2.moveWindow("Hit Analysis", 400, 50) print("System ready.")
while True: window = np.zeros((600, 800, 3), dtype=np.uint8) current_time = time.time() is_connected =
(current_time - sensor_data["last_update"]) < 1.0
if is_connected: HUD = cv2.rectangle(window, (10, 10), (240, 590), (20, 20, 20), -1); cv2.rectangle(window, (10, 10), (240, 590), (10,
if waiting_for_snapshot1: curr_acc = history_acc[-1] if history_acc else 0; curr_force_n = history_force_n[-1] if history_force_n else 0; curr_gyro =
history_gyro[-1] if history_gyro else 0
if curr_acc > peak_acc_current: peak_acc_current = curr_acc; if curr_force_n > peak_force_n_current:
peak_force_n_current = curr_force_n; if curr_gyro > peak_gyro_current: peak_gyro_current = curr_gyro
HUD Display (Session Peak Values) disp_acc = peak_acc_current; if waiting_for_snapshot1: disp_force_n = peak_force_n_current; if waiting_for_snapshot1: disp_gyro = peak_gyro_current; if waiting_for_snapshot1: disp_gyro = peak_gyro_current
1. ACCELERATION cv2.putText(window, "MAX ACC:", (25, 100), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1); cv2.putText(window, "FORCE (NEWTONS) - NEW", (25, 200), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1); cv2.putText(window, "ROTATION", (25, 300), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1); cv2.putText(window, "GRAPHS (Now 3 pieces)", (25, 400), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1)
3.0, title = "ACCELERATION(Acc)", unit = "m/s^2", grid_step = 10)
2. FORCE N (Middle, Orange) - Smaller scale because values are large (e.g. 40g * 4kg = 160N)
draw_graph(window, list(history_force_n), 260, 200, 520, 160, (0, 165, 255), scale_factor = 0.8, title =
"HIT FORCE (F = ma)", unit = "N", grid_step = 50)
3. ROTATION (Bottom, Purple) draw_graph(window, list(history_gyro), 260, 380, 520, 160, (255, 0, 255), scale_factor =
0.3, title = "ROTATION(Gyro)", unit = "deg/s", grid_step = 100)
-- HIT DETECTION -- acc_now = history_acc[-1] if history_acc else 0
if acc_now > THRESHOLD_HIT and not waiting_for_snapshot1 and not waiting_for_snapshot2:
shot_rigger_time = current_time; Reset peaks at start of new session; peak_acc_current = acc_now; peak_force_n_current =
acc_now * ESTIMATED_ARM_ASS; peak_gyro_current = 0.0
waiting_for_snapshot1 = True; waiting_for_snapshot2 = True
Snapshot 1 (MOMENT OF IMPACT) if waiting_for_snapshot1 and (current_time - shot_rigger_time >
DELAY_SNAPSHOT1):
hist_a = list(history_acc); hist_f = list(history_force_n); hist_g = list(history_gyro)
if hist_a: INTELLIGENT SEARCH FOR 2ND PEAK (Skipping swing) peaks_indices = [] for i in range(1, len(hist_a)-
1): if hist_a[i] > THRESHOLD_PEAK: if hist_a[i] >= hist_a[i-1] and hist_a[i] >= hist_a[i+1]:
peaks_indices.append(i)
impact_dx = 0 if peaks_indices: if len(peaks_indices) > 1: candidates = peaks_indices[1:]
Discard 1st peak (swing) impact_dx = max(candidates, key=lambda i: hist_a[i]) Highest of the rest else:
impact_dx = peaks_indices[0] else: impact_dx = np.argmax(hist_a)
-- GET VALUES FROM IMPACT POINT -- saved_acc = hist_a[impact_dx]
if impact_dx < len(hist_f): saved_force_n = hist_f[impact_dx] else: saved_force_n = peak_force_n_current
if impact_dx < len(hist_g): saved_gyro = hist_g[impact_dx] else: saved_gyro = peak_gyro_current
Draw Line graph width = 520; x_start = 260; step = graph_width/len(hist_a); impact_x = x_start +
int(impact_dx*step); cv2.line(window, (impact_x, 20), (impact_x, 540), (255, 255, 255), 1); cv2.putText(window, "IMPACT", (impact_x,
5, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 255, 255), 1)
snapshot = window.copy() Saving only this one image; threading.Thread(target=save_file, args =
(saved_acc, saved_force_n, saved_gyro, snapshot), daemon=True).start(); waiting_for_snapshot1 =
False
End of session (without saving second image) if waiting_for_snapshot2 and (current_time - shot_rigger_time >
DELAY_SNAPSHOT2): waiting_for_snapshot2 = False

```

```

if waiting_for_snapshot1 : cv2.putText(window,"ANALYZING...",(30,550),cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,255,255),2)
else: cv2.putText(window,"WAITING...",(200,300),cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,0,255),2)
cv2.imshow("Hit Analysis", window) if cv2.waitKey(20) & 0xFF == ord('q'): break
cv2.destroyAllWindows()

if name=="__main__":
    List of additional files in electronic submission
    main()

```

The electronic submission attached to this thesis contains a compressed archive (.zip) organized as follows:

- **Source Code:** The complete source code of the system, divided into:
 - /pc_server – Python scripts for the Vision and Telemetry modules (bilard.py, sensors.py) including the req.txt file.
 - /android_app – The Android Studio project files for the mobile sensor application.
- **Executables:**
 - SensorApp.apk – The compiled installation package for the Android mobile device.
- **Configuration:**
 - config.json – Default configuration file with calibration parameters and anthropometric data structure.
- **Documentation:**
 - README.md – Short instruction on how to run the environment and install dependencies.

[width=0.8]./graf/cue-detection/confusion_matrix.pdf

Figure 32: Confusion Matrix for Cue Detection, showing 70 correct detections against 6 false negatives (background).

[width=0.85]./graf/soft.png

Figure 33: Soft Shot analysis: Acceleration $< 20 \text{ m/s}^2$. The system treats this as noise/preparation and does not register a shot.

[width=0.85]./graf/medium.png

Figure 34: Medium Shot analysis: Distinct peak detected. Calculated Force = 54 N.

[width=0.85]./graf/hard.png

Figure 35: Hard Shot analysis: High-intensity peak. Calculated Force = 97 N.